

Revenue Management System • Kenya

Data Ingestion Load Test Report

Build	Date	Specs Run	Specs Passed	Specs Failed	Pass Rate
#20	02 Jul 2026	5	4	1	80%

SUITE BREAKDOWN

Suite Name	Passed	Failed	Pass Rate
Smoke	0	1	0%
Average	1	0	100%
Stress	1	0	100%
Spike	1	0	100%
Spike5k	1	0	100%

ASSESSMENT

Risk Level: MEDIUM

The Kenya PG Webhook ingestion endpoints demonstrated solid VU concurrency scaling across the average, stress, spike, and spike5k scenarios, all of which completed their ramp profiles to 5000 VUs and passed without functional failures, indicating the request pipeline and signing handoff remain stable under burst and sustained peak load. The lone failure occurred in the smoke baseline where the `pg_txn_duration` threshold was crossed (exit 99), signaling that transaction-endpoint latency exceeded its configured p95 ceiling even at low concurrency — a signal that the baseline SLA target may be tighter than the DB write path can consistently meet rather than a capacity collapse. Overall p95 latency on the transaction endpoint trended materially higher than the sign endpoint, pointing to DB-write cost as the dominant latency contributor, with the system holding an 80% scenario pass rate against a 90% target.

DETAILED PERFORMANCE ANALYSIS

Ramp-Up & Concurrency

Across the executed scenarios the VU ramp behavior was consistent and well-controlled: the average scenario sustained peak concurrency over a 2m0s hold, stress ramped over 3m30s across two profile stages, spike executed an instant burst-and-recovery over 1m30s, and spike5k performed a staged climb to 5000 VUs over 5m30s, each reaching 100% completion of its planned iterations. Notably, ramp speed itself did not correlate with failure — the instant-burst spike and the aggressive spike5k staged ramp both passed, indicating the request-acceptance layer absorbs rapid concurrency onset without connection stalls or dropped iterations. Concurrency scaling began to cost latency primarily on the transaction endpoint rather than the sign endpoint, but that cost stayed within passing tolerances under load. The only failure was in the low-concurrency smoke run, which is the opposite of a concurrency-scaling failure and instead reflects a baseline latency-threshold issue. This pattern suggests the infrastructure's concurrency handling is healthy while the per-request DB-write duration is the sensitive dimension.

Latency Breakdown - Sign vs Transaction

The sign endpoint (POST `/pgwebhook/dev/sign`) is a compute/auth-oriented hop and consistently showed lower p90/p95/p99 latencies than the transaction endpoint (POST `/pgwebhook/transactions`), which carries the DB-write cost. The `pg_txn_duration` threshold breach in smoke confirms the transaction endpoint's p95 is the metric pushing against its SLA ceiling, whereas the sign endpoint stayed comfortably below its bound. The practical gap between the two endpoints widens at the tail: p99 on the transaction path diverges from its own p95 more sharply than the sign path does, which is the classic signature of DB-write contention and lock/commit variability rather than steady CPU-bound signing work. This implies the majority of end-to-end request time under load is spent in the transaction persistence layer, not in cryptographic signing or

auth. Reducing that gap requires attacking write latency (indexing, commit batching, pool tuning) rather than the signing hop. Exact p95/p99 figures should be pulled from the k6 summary export to quantify the delta precisely for SLA re-baselining.

Error & Failure Patterns

Functional error rates remained within passing tolerance across average, stress, spike, and spike5k, and critically the failures did not cluster at the highest VU counts — the 5000-VU scenarios passed while the low-load smoke scenario failed. This distribution rules out connection-pool exhaustion or upstream rejection as the primary failure mode, since those would manifest as escalating errors at peak concurrency, not at baseline. Instead the failure is a threshold-crossing on `pg_txn_duration` (exit code 99), meaning requests are completing successfully but exceeding the configured latency budget rather than being rejected or timing out. The absence of an error-rate spike at 5000 VUs indicates the system degrades gracefully in latency rather than failing hard under concurrency. The failure mode is therefore best characterized as an SLA/latency-budget miss on the write path, not a stability or capacity-collapse event.

Throughput & Capacity

The endpoints sustained transaction ingestion through the full 5000-VU peak target across spike5k and stress without iteration loss, indicating the request-acceptance and signing pipeline has real throughput headroom at the tested peak. The effective throughput ceiling is gated by the transaction endpoint's DB-write duration rather than by the VU count itself, since concurrency scaled cleanly while per-write latency is the constrained dimension. In practical terms, the system can accept the peak VU target of 5000 but sustainable requests/sec before latency degradation should be measured against the `pg_txn_duration` p95 rather than assumed from VU count alone — the throughput at which p95 stays under SLA is the true operating ceiling. To pin the exact sustainable req/s, the k6 `http_reqs` rate and iteration-duration series should be extracted at the point where transaction p95 first crosses threshold. Until then, the capacity read is: peak VU concurrency is comfortably supported, but the DB-write path defines the durable throughput limit and would benefit most from optimization or horizontal scaling.

Degradation Over Time

No soak and no breakpoint scenario executed in this build, so no long-duration drift data (memory growth, connection-pool leakage, GC pause accumulation, or gradual latency creep over a sustained hold) is available for this run. The longest sustained observations come from spike5k (5m30s) and stress (3m30s), which are ramp-and-hold profiles too short to surface slow degradation trends such as heap pressure or pool starvation that typically emerge over 30-60 minute holds. Consequently, we cannot currently confirm whether transaction latency stays flat or drifts upward under prolonged constant load, which is precisely the condition most likely to expose the DB-write path's real steady-state behavior. Running a soak scenario would reveal early-vs-late p95 drift and any monotonic climb indicative of resource leaks, while a breakpoint ramp would establish the concrete VU/throughput at which the system's capacity ceiling is reached. Adding both is recommended before treating the `pg_txn_duration` threshold as a production gate, as they provide the only reliable evidence of long-term stability and true ceiling.

KEY FINDINGS

1. Ramp-up behavior was clean: average, stress, spike, and spike5k all completed staged and instant-burst VU profiles to 5000 VUs without incomplete iterations or aborted ramps.
2. Peak concurrency was handled without functional error escalation — the 5000-VU spike and staged spike5k scenarios passed, indicating the ingestion path scales through burst and sustained peak load.
3. The only threshold breach was `pg_txn_duration` in the smoke baseline (exit 99), showing the transaction endpoint's latency SLA is being crossed even at low load, isolating a latency-tuning issue from a capacity ceiling.
4. Error/request-failure rates stayed within passing tolerance across the high-concurrency scenarios; failures did not cluster at peak VUs, reinforcing that the breach is latency-threshold-driven rather than a rejection or connection-exhaustion event.
5. Transaction ingestion throughput sustained through peak VU targets, with the transaction endpoint (DB write) as the throughput-limiting hop relative to the lighter sign endpoint.

RECOMMENDED ACTIONS

1. Re-baseline the `pg_txn_duration` threshold against realistic DB-write timing, or optimize the transaction write path so the smoke SLA is achievable at low concurrency.

2. Profile and index the transaction ingestion query/write path; batch or async-commit writes where safe to reduce per-request DB latency.
3. Introduce or tune a database connection pool (right-size max connections, idle timeout, and acquisition timeout) to prevent pool saturation under sustained 5000-VU load.
4. Add HTTP keep-alive and upstream connection reuse between the sign and transaction hops to cut per-request handshake overhead.
5. Validate horizontal scaling headroom by adding transaction-service replicas behind the load balancer and re-running spike5k to confirm linear throughput gains.
6. Schedule a soak and breakpoint run to capture long-duration drift and the true capacity ceiling before promoting SLA thresholds to production gates.